Intern Onboarding Plan — Inventory Tracker (Warehouse)

Oct 28, 2026 · Mentor: @RapidM3

Welcome and goal

In 4 days you will set up Claude Code with our team configuration and build a small full-stack app on our stack: an **Inventory Tracker for a warehouse**. Real tasks start once you finish.

The app: warehouse staff sign in, search products by name or SKU, filter by category, see current stock and low-stock items, open a product's movement history, and record stock in or out. Data lives in MySQL behind a Spring Boot API, with a React frontend and Firebase login.

Your teammate does the same job on the same stack in a different context: an Employee Directory for an HR team. On Day 4 you each fix a bug in the other's repo.

You are done when:

1. The app (database, API and UI) runs from a fresh clone using only your README.
2. You have opened 2 pull requests, both reviewed by your mentor: your project, plus a fix in your teammate's repo.
3. You can explain every line you submitted without asking Claude to explain it for you.

Ask your mentor before you expand scope. Flag blockers within 30 minutes; don't sit on them.

Our tech stack

You touch every layer of our stack. Each row shows what that layer does in your Inventory Tracker.

Layer	Technology	In your app
Database	MySQL 8, SQL	`category`, `product` and `stock_movement` tables
Backend API	Java 17+, Spring Boot, Spring Web, Spring Data JPA, Bean Validation	REST endpoints for products, categories and stock movements
Testing	JUnit, Spring Boot Test	Proves the API rules work

Layer	Technology	In your app
Frontend	React (JS/TS), Vite, CSS	Product list, search, filter, low-stock view, details, movement form
Identity	Firebase Authentication	Warehouse staff sign in before seeing any data
Tooling	Git, feature branches, pull requests	Every change is reviewed
AI assistant	Claude Code + team setup	Explore, plan, implement, verify

Key idea: hiding a page in React is not security. Only the backend can enforce access, by verifying the Firebase ID token on every protected request.

How to use Claude Code

Treat Claude as a fast pair programmer whose every suggestion you check. You own the correctness, security and quality of what you submit.

The 4 rules in our CLAUDE.md (Claude follows them; you should too)

1. **Think before coding.** State assumptions and ask when something is unclear.
2. **Simplicity first.** Write the minimum code that solves the task, with no speculative features.
3. **Surgical changes.** Touch only what the task needs and match the existing style.
4. **Goal-driven execution.** Define a check (test, run, query), then loop until it passes.

Plugins and when to use them

Plugin / skill	Use it for
superpowers: `brainstorming`	Before designing any feature, schema or component
superpowers: `writing-plans`	Turning an agreed design into steps you review
superpowers: `test-driven-development`	Writing a failing test first, then the code
superpowers: `systematic-debugging`	Any bug: reproduce, gather evidence, then fix
superpowers: `verification-before-completion`	Before you say anything is done

Plugin / skill	Use it for
`/code-review`	Reviewing your own diff before opening a PR
code-simplifier	Trimming code Claude over-built
context7	Current docs for React, Firebase, Spring, so Claude doesn't invent APIs
playwright	Clicking through your UI in a real browser
frontend-design	UI layout and styling decisions
explanatory output style	Claude explains its choices as it works. Keep it on while learning

Off-limits until your mentor says otherwise: ruflo / claude-flow swarms and multi-agent commands, `ralph-loop`, the Atlassian and other connected-account tools, and deploying anywhere other than your own machine, touching a shared or production database, or pushing to `main`.

Prompt pattern for every task

```
Explore this repo and explain <area>. Do not edit files.
Then propose a small plan: files affected, assumptions, acceptance criteria, tests.
Wait for my approval before implementing.
```

Day 1: Setup, stack tour and design

Goal: Claude Code working, every layer of the stack running locally, and an approved design for your app.

Morning: environment and Claude Code warm-up

- [ ] Install JDK 17+, Maven, MySQL 8 (local), Node.js LTS and Git. Create your GitHub repo `inventory-tracker` with `api/` and `web/` folders.
- [ ] Warm-up: in any existing team repo, ask Claude to explain the folder structure, entry point and tests **without editing**. Write 5 things you learned in your log.
- [ ] Use `context7` to pull current Spring Boot, React and Firebase Auth docs into a session. Note one API Claude got wrong before it checked the docs, if any.

Afternoon: stack tour and design

- [ ] Generate the API with Spring Initializr (Web, Data JPA, Validation, MySQL Driver) and the frontend with Vite (React). Run both empty apps.

- [ ] Create a **personal dev** Firebase project and enable Email/Password sign-in.
- [ ] Decide with your mentor how current stock is calculated: the sum of a product's stock movements (recommended), not a separately edited number.
- [ ] Run `superpowers:brainstorming` with this prompt, then draw the ER diagram and component tree yourself:

```
Help me design a small full-stack Inventory Tracker for a warehouse.
Data: categories, products (name, SKU, category, reorder level), and
stock_movements
(product, quantity change, type IN/OUT/ADJUST, timestamp); current stock is
the sum
of a product's movements. Propose the MySQL tables, keys, constraints and
indexes,
the REST endpoints, and a React component tree with search, category filter,
low-stock view, details with movement history and a record-movement form,
behind
Firebase email/password login. List assumptions and trade-offs. Do not write
code
or run SQL. Wait for my approval.
```

End of day check-in (15 min with mentor): show the ER diagram, the endpoint list and the component tree. Get approval before Day 2.

Day 2: Database and API

Goal: the database builds from scripts, and a tested, validated API serves it.

Morning: MySQL

- [ ] Create a feature branch: `git checkout -b feat/api`.
- [ ] Write `db/schema.sql` with `category`, `product` and `stock_movement`: primary keys, foreign keys, `NOT NULL` where required, a unique SKU, and only indexes you can justify by a query.
- [ ] Write `db/seed.sql` with fake data: 4 categories, 15+ products, and IN/OUT movements for each.
- [ ] Run both scripts on a **fresh local database**. Then insert a movement for a non-existent product and a product with a duplicate SKU, and confirm both are rejected.
- [ ] Write `db/queries.sql` with 5 commented queries: products by category, current stock per product, one product's movement history, low-stock products (stock below reorder level), and movement totals per type. Check 2 results by hand.

Afternoon: Spring Boot

- [ ] Put DB credentials in environment variables or a git-ignored `application-local.properties`. Commit only an example file.
- [ ] Build one layer at a time (entity, DTOs, repository, service, controller), committing after each.
- [ ] Endpoints: `GET /products?name=&categoryId=` (with current stock), `GET /products/{id}`, `POST /products`, `GET /products/low-stock`, `POST /products/{id}/movements`, `GET /categories`.
- [ ] Return `400` for invalid input, `404` for unknown IDs and `409` for a duplicate SKU or a movement that would take stock below zero, with a clear JSON error body.
- [ ] Use `superpowers:test-driven-development`: a failing JUnit test per rule above, then make it pass. Run `mvn test` yourself.

End of day check-in: rebuild the DB from scripts, run `mvn test`, and show a curl request for each error case.

Day 3: React UI and Firebase login

Goal: signed-in warehouse staff can use the whole tracker in the browser; signed-out users see nothing.

Morning: React UI on the real API

- [ ] Create a branch `feat/web`. Ask Claude to turn the approved component tree into a plan (`superpowers:writing-plans`) and review it.
- [ ] Build one component at a time, committing after each: `ProductList`, `ProductRow`, `SearchBox`, `CategoryFilter`, `LowStockView`, `ProductDetails` (with movement history), `MovementForm`.
- [ ] Fetch from your API (enable CORS for the Vite dev URL only). Search and category filter must work **together**.
- [ ] Handle the states: loading, empty results, and API errors such as "stock would go below zero" shown on the form.
- [ ] For each component, ask Claude to explain its props, state and hooks. Write one line per component in your log.

Afternoon: Firebase Auth

- [ ] Ask Claude to explain the Firebase Auth flow and `onAuthStateChanged` **before** it writes code.
- [ ] Put the Firebase web config in `web/.env.local` (Vite `VITE_` variables), git-ignore it, and commit a `.env.example`.

- [ ] Build Register, Login and Logout. Protect every tracker route; signed-out users go to Login.
- [ ] Show plain messages for wrong password, unknown user, email in use and weak password.
- [ ] Stretch: send the Firebase ID token to the API and verify it in Spring with the Firebase Admin SDK. If you skip it, be ready to explain how it would work.
- [ ] Check layout at desktop and mobile widths with `playwright` screenshots.

End of day check-in: register, sign in, search + filter, record a stock OUT, check the low-stock view updates, sign out, and confirm the tracker is unreachable.

Day 4: Cross-repo fix, PRs and demo

Goal: practise the real-work loop on code you didn't write (your teammate's Employee Directory), then ship and present your own app.

Morning: fix one bug in your teammate's Employee Directory

Your mentor picks one small, approved issue in the teammate's repo (for example, filters not combining or a missing error message).

- [ ] Clone the repo, build its database from the scripts on your **local** MySQL, and run the API and UI.
- [ ] Reproduce the bug before you change anything.
- [ ] Use `superpowers:systematic-debugging` with this prompt:

```
I have a mentor-approved bug to investigate. Inspect the relevant code and
explain
the execution path without modifying files. Given my reproduction steps, list
likely
root causes and how to tell them apart. Propose a minimal fix and a
regression test,
but wait for my approval before editing.
```

- [ ] Write a failing regression test, apply the smallest fix, and confirm all tests pass.
- [ ] Open a PR covering problem, root cause, fix and test output. Your mentor reviews it.

Afternoon: ship and present

- [ ] Finish your README: prerequisites, DB setup, env variables, Firebase steps, how to run the API, UI and tests.
- [ ] Rebuild the DB from scripts one last time, run `mvn test`, and run your manual test checklist.

- [ ] Run `superpowers:verification-before-completion` and `/code-review` on the full diff, then open your PR.
- [ ] Your mentor reviews your PR. Address every comment before the final demo.
- [ ] Give your final demo to your mentor (see Final demo below).

Deliverables, review and rules

Deliverables

- [ ] `inventory-tracker` repo: `db/` scripts, `api/` and `web/`, all running from the README.
- [ ] `db/schema.sql`, `db/seed.sql` and `db/queries.sql` (5 commented queries) that run on a clean MySQL database, plus an ER diagram and a note on how stock is calculated.
- [ ] API with validation and 400/404/409 handling; JUnit tests runnable with `mvn test`.
- [ ] React UI with search + category filter, low-stock view, details with movement history, movement form, loading/empty/error states, and Firebase login protecting every page.
- [ ] Manual test checklist with results: filter combinations, empty results, duplicate SKU, stock below zero, mobile width, register, wrong password, logout, protected page while signed out.
- [ ] Mentor-reviewed PR fixing one bug in your teammate's Employee Directory, with a regression test.
- [ ] A short Claude Code note: prompts you used, and at least one suggestion you accepted, changed or rejected, and why.

Final demo (end of Day 4, 30 min with your mentor)

Run everything locally from a fresh clone and walk your mentor through:

1. Rebuild the database from your scripts, then start the API and UI (2 min).
2. Live walkthrough: register, sign in, search + category filter, open a product's movement history, record a stock OUT, show the low-stock view update, trigger a below-zero error, sign out (8 min).
3. Show `mvn test` passing and your manual test checklist results (3 min).
4. Walk through your PR and one design decision you would defend (5 min).
5. Your fix in the Employee Directory: the bug, root cause and regression test (5 min).
6. Questions (7 min).

Be ready to answer:

1. Trace one request from the React search box to MySQL and back.
2. Why separate controller, service and repository? What is an entity versus a DTO?

3. How is current stock derived from movements, and why not store a single editable number?
4. What do your foreign keys and unique SKU prevent? Which index did you add, and why?
5. How does the UI know a user is signed in, and why is that not enough to protect the API?
6. Which Firebase config values are safe in a web client, and which secrets are not?
7. In your teammate's repo: what was the root cause, and what does your regression test prove?
8. Which Claude suggestion did you reject or change, and why?

Rules

- Never paste passwords, tokens, private keys or real customer data into Claude.
- Never commit `.env` files, DB passwords or credentials.
- Only run SQL against your local database. No shared or production databases.
- Run, test and deploy everything locally. Nothing goes to a shared or cloud environment without mentor approval.
- Don't disable tests or weaken validation to make a build pass.
- Don't report a test as passing unless you ran it and read the output.

Daily log (fill in at the end of each day)

Day	What I completed	What I verified / learned	Claude suggestion I changed or rejected
1			
2			
3			
4			